

Project 1: Graph Data Processing

CSE 511: Data Processing at Scale

Introduction

Graph data appears in the web, biology, transportation, and distributed systems. In this project, you will build a **Dockerized Neo4j** environment, load a real dataset, install the **Graph Data Science (GDS)** plugin, and implement two classic graph algorithms: [PageRank](#) and [Breadth First Search \(BFS\)](#).

Step 0 — Exploring the world of Graph Databases (0 pts)

1. Create a **free Neo4j account** to start exploring! They provide a variety of tools for developers, and in this part of the project, we are going to explore one of those tools: [Neo4j Aura](#).
2. Create a new AuraDB **instance** (only the first instance is free). Go ahead and create that instance and load the provided Stack Overflow database.
3. Load the Stack Overflow sample and complete the guided tasks in the right-hand toolbar to learn Cypher and graph exploration.

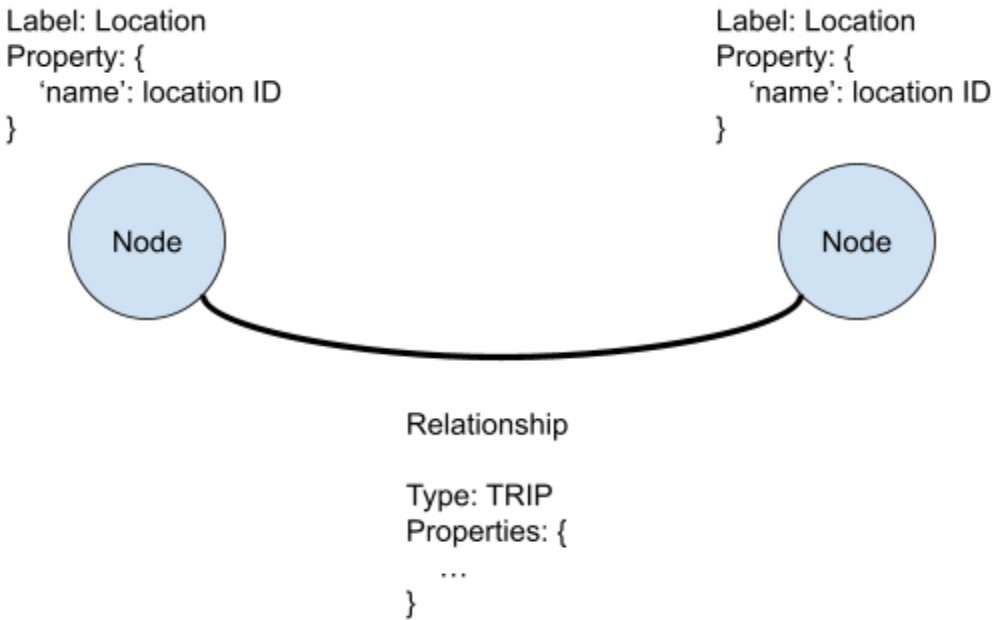
Step 1: Dockerfile & Data Loading (50 pts)

In this step, you will **build a Docker container** that installs Neo4j and **loads the [NYC taxi dataset](#)** into it. The schema of the NYC Trip dataset is available [here](#). You can also go through this short [blog](#) to understand how such datasets are making cities smarter.

You will learn how a **Dockerfile** is structured and how to set up your working environment. By the end of this step, you should be able to **build and run your own Neo4j container**, verify that the data has been loaded correctly, and confidently explore the database inside your container.

To get you started, we have already provided you with a template `Dockerfile` [here](#) that performs various preliminary tasks like setting up the operating system and the target build platforms (so that your container can run on both ARM and x86). We have also provided you with a template `data_loader.py` file [here](#). You need to write a query to load the data into the file. The schema you should follow is described below:

- **Nodes**
 - Label: `Location`
 - One node per pickup/dropoff `locationID`
 - Property: `name (locationID, integer)`, taken from `PULocationID` and `DOLocationID`



- **Relationships**

- Type: **TRIP**
- One relationship per trip
- Properties:
 - **distance** (float)
 - **fare** (float)
 - **pickup_dt** (datetime)
 - **dropoff_dt** (datetime)

Important: Use **exact** labels and property names above. Datetimes may include a timezone or not; grading accepts either.

Hints: These are some high-level steps to implement Step 1. **What your Dockerfile must do:**

1. Create a working directory:
 - a. Path: `/cse511/`
2. Download the [March 2022 NYC Taxi dataset](#) to the appropriate location inside the container.
3. Implement `data_loader.py` (before building the Docker image) and **copy** it from the same directory as your Dockerfile (relative path `./`) into `/cse511/` in the container.
 - a. The `data_loader.py` script will be executed in the RUN block later (see the end of the Dockerfile template).
 - b. Most of the setup logic has been provided in the `data_loader.py`, so your main task is to **load the data into Neo4j following the specified schema**.

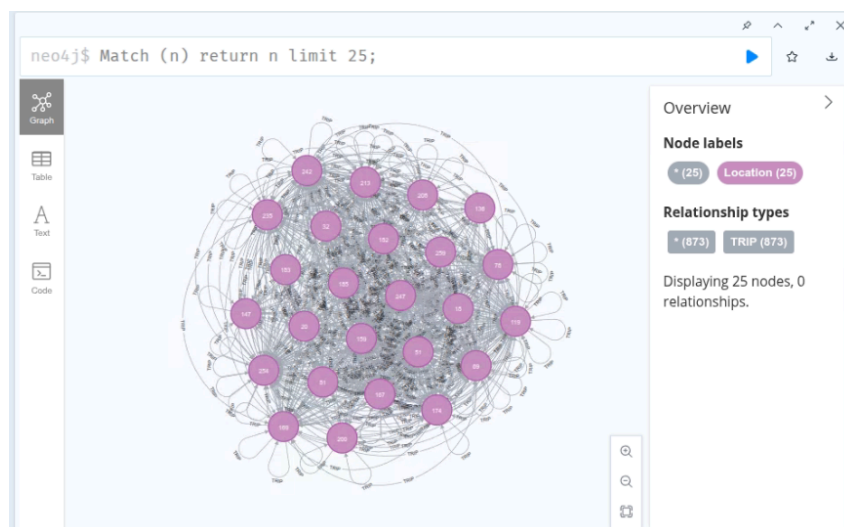
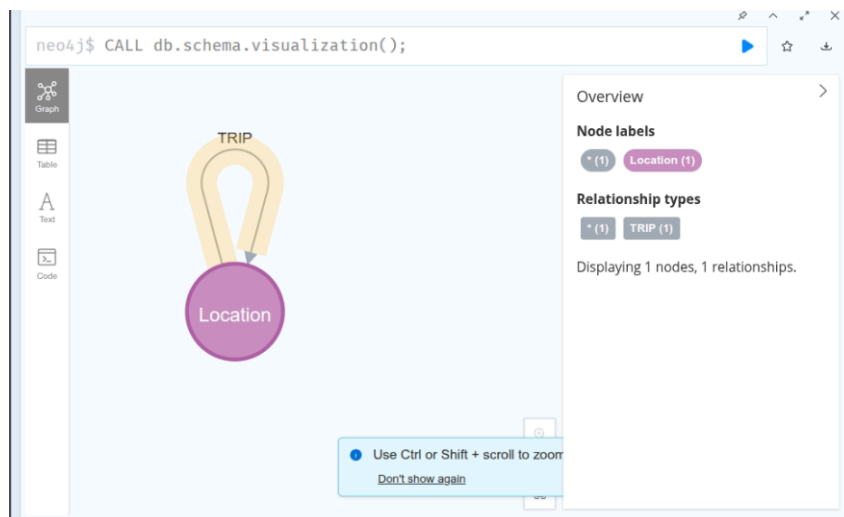
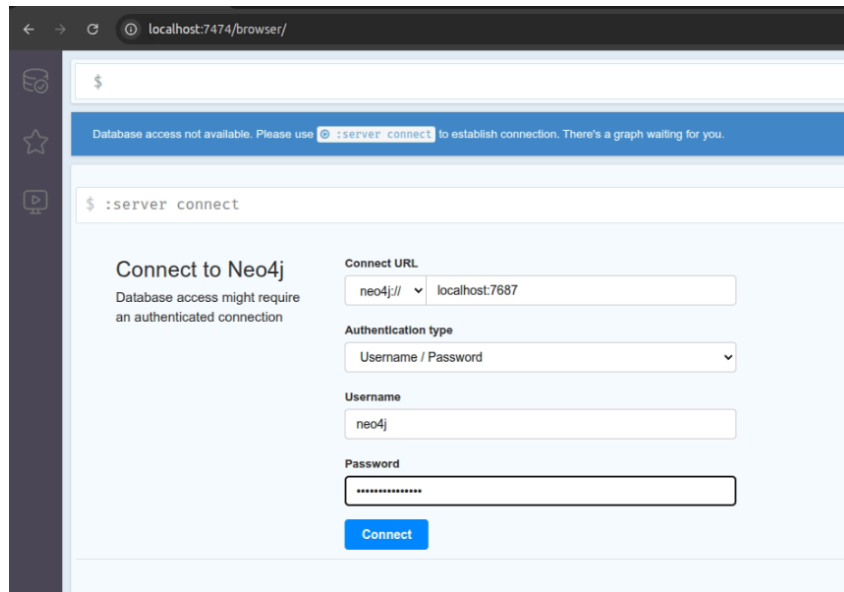
4. Install Java (OpenJDK 21):
 - a. Run apt update and install openjdk-21-jdk.
 - b. Set the environment variable JAVA_HOME `/usr/lib/jvm/java-21-openjdk-amd64`
5. Upgrade pip and install only the required Python packages: `neo4j`, `pandas`, `pyarrow`
6. The GDS (graph data science) plugin adds several basic and useful procedure calls to Neo4j, and more specifically, it adds procedure calls for Step 3. Download and install the Neo4j Graph Data Science (GDS) plugin v2.21.0:
 - a. Get the .jar file from: <https://github.com/neo4j/graph-data-science/releases/>
 - b. Move it to the plugin directory: `/var/lib/neo4j/plugins/`
7. Configure `/etc/neo4j/neo4j.conf` non-interactively by appending the required settings during the build process. You can refer the [this document](#) to get started with how to configure `Neo4j` for machines that are not local. Docker, in some way, behaves like a remote machine for this particular scenario. You can consider that and try those changes.
 - a. Allow Neo4j to listen on all network interfaces (0.0.0.0) so it can be accessed from the host.
 - b. Whitelist and enable unrestricted access to the APOC and GDS procedures.
 - c. Make these configuration updates automatically within the Docker build (for example, by appending lines to the neo4j.conf file).
8. Set the default Neo4j password to `graphprocessing`.

If you have followed all the steps correctly, you will be able to create a Docker image using your Dockerfile, and on running a container using this instance, your `Neo4j` instance will be live.

Then, your container is ready to perform operations. We have exposed port 7474 from inside Docker, which allows you to even browse `localhost:7474` on your host machine and interact with your Neo4j browser directly! Some queries you might want to try running over there:

- Viewing the DB schema: `CALL db.schema.visualization();`
- View some of the data: `MATCH (n) RETURN n LIMIT 25;`

Note: Be patient after creating your container. It takes 2-4 minutes for the server to be available after starting.



Step 2: Implementing Graph Algorithms (50 pts)

With your Neo4j container up and the dataset loaded, implement the two algorithms below in [interface.py](#). The `interface.py` file does not need to be inside the container and will connect to the Docker container from outside it. I.e., using the 7687 port that we have exposed from inside Docker.

Note: Do not copy `interface.py` or `tester.py` into your Dockerfile build. During grading, the grader will only build your image using your Dockerfile and `data_loader.py` from the same folder—no other files are included. Bundling `interface.py` or `tester.py` in the image will cause grading errors.

1. [PageRank](#) (25 pts)

The PageRank algorithm evaluates the significance of each node in a graph by considering the number of incoming relationships and the importance of the source nodes that contribute to it. The basic premise of this algorithm is that a page's importance is determined by the pages that link to it. The page rank in the original [Google paper](#) is defined as below

$$PR(A) = (1 - d) + d \left(\frac{PR(T_1)}{C(T_1)} + \dots + \frac{PR(T_n)}{C(T_n)} \right)$$

Where

- We assume that page A has pages T_1 to T_n , which point to it.
- d is a damping factor which can be set between 0 (inclusive) and 1 (exclusive). It is usually set to **0.85**.
- $C(A)$ is defined as the number of links going out of page A.

This equation is used to iteratively update a candidate solution and arrive at an approximate solution to the same equation.

What to implement?

You need to implement the PageRank algorithm, which will tell you how the locations (nodes) you have created in the graph fare against each other. Make sure that you take the `max_iter` and `weight_property` variables into consideration. A helpful guide here will be the official GDS documentation for PageRank. Once done, you need to return the nodes with the maximum and minimum PageRank.

2. [Breadth First Search](#) (25 pts)

The Breadth First Search algorithm is a graph traversal algorithm that, given a start node, visits nodes in order of increasing distance. When the probability of finding a specific node decreases as the distance increases, this algorithm is useful for searching. It

supports various termination conditions for the traversal, such as reaching one of several target nodes, reaching the maximum depth, exhausting a given budget of traversed relationship cost, or traversing the entire graph.

What to implement?

You need to implement the BFS algorithm from location A to multiple locations. The locations will be provided using 2 variables: start_node and target_nodes. Make sure to use these variables.

Note: The Neo4j GDS library has support for these algorithms; you are permitted to install the library and use the algorithms directly.

Grading:

- We expect to use your Docker file to create an image and deploy your container to test your code (interface.py). The Docker file **MUST** be able to perform the following steps **without any manual intervention**. (Failure to do so will result in 0 grade points)
- **DO NOT EDIT** any existing code in the template Docker file.

Submission Requirements & Guidelines

1. **Individual work:** This is an **individual assignment**; all work must be your own.
2. **What to submit on the canvas?**
Create a **zip** file containing the following:
 1. Dockerfile
 2. interface.py
 3. data_loader.py
3. **What is the Submission File nomenclature?**
 1. Name the **zip** as per your ASU 10-digital ID; e.g., 1221234567.zip
 2. A different filename or submitting additional files will incur a **30-point penalty**.

Submission Policies

1. Late submissions will **absolutely not** be graded (unless you have verifiable proof of an emergency). It is much better to submit partial work on time and get partial credit for your work than to submit late for no credit.
2. Every student needs to **work independently** on this exercise. We encourage high-level discussions among students to help each other understand the concepts and principles. However, a code-level discussion is prohibited and plagiarism will directly lead to failure of this course. We will use anti-plagiarism tools to detect violations of this policy.

Grading Rubric (100 pts)

Part 1: Dockerfile & Data Loading (50 points)

- 10 pts: Docker image builds successfully without errors
- 5 pts: Working directory /cse511/ created
- 5 pts: Java (OpenJDK 21) installed
- 5 pts: Python packages installed (neo4j, pandas, pyarrow)
- 15 pts: Neo4j Configuration
- 10 pts: Data Loading

Part 2: PageRank Algorithm (25 points)

- 5 pts: each test case

Part 3: Breadth First Search (25 points)

- 5 pts: each test case

Testing Commands

Build

```
docker build -t project1 .
```

Run

```
docker run -d -p 7474:7474 -p 7687:7687 --name p1 project1
```

Test (wait 2-4 minutes for Neo4j startup)

```
python3 tester.py
```

Deductions

- -100 pts: Manual intervention required during build
- -100 pts: Modified template code
- -50 pts: Container doesn't start or Neo4j service fails
 - When the grader runs: `docker run -d -p 7474:7474 -p 7687:7687 <your_image>`
- -30 pts: Incorrect filenames or extra files submitted

```

chang@chang-pc:~/docker-file/project1$ python3 tester.py

=====
PROJECT 1: GRADER VERSION - COMPREHENSIVE TESTS
=====

⚠ WARNING: This is the grader version with multiple test cases
to prevent hardcoded solutions.

Connecting to Neo4j server ✓
Server is running!

=====
PART 1: DATA LOADING TESTS
=====

Node Count Test (5 points):
  Expected: 42 nodes
  Actual:   42 nodes
  ✓ PASS [5/5 points]

Edge Count Test (5 points):
  Expected: 1530 edges
  Actual:   1530 edges
  ✓ PASS [5/5 points]

-----
Part 1 Score: 10/10 points
-----

=====
PART 2: PAGERANK TESTS (GRADER VERSION)
=====

[Test 1/5] PageRank with max_iterations=20, weight='distance'
  ✓ PASS

[Test 2/5] PageRank with max_iterations=10, weight='distance'
  ✓ PASS

[Test 3/5] PageRank with max_iterations=5, weight='distance'
  ✓ PASS

[Test 4/5] PageRank with max_iterations=15, weight='distance'
  ✓ PASS

[Test 5/5] PageRank with max_iterations=25, weight='distance'
  ✓ PASS

  ✓ ALL TESTS PASSED [25/25 points]

-----
Part 2 Score: 25/25 points (5/5 tests passed)
-----

=====
PART 3: BFS TESTS (GRADER VERSION)
=====

[Test 1/5] BFS from node 159 to node 212
  ✓ PASS

[Test 2/5] BFS from node 159 to node 167
  ✓ PASS

[Test 3/5] BFS from node 3 to node 240
  ✓ PASS

[Test 4/5] BFS from node 159 to node 200
  ✓ PASS

[Test 5/5] BFS from node 200 to node 81
  ✓ PASS

  ✓ ALL TESTS PASSED [25/25 points]

-----
Part 3 Score: 25/25 points (5/5 tests passed)

```



```
=====
FINAL SCORE SUMMARY (GRADER VERSION)
=====
```

```
Part 1 - Data Loading:    10/10 points
Part 2 - PageRank:       25/25 points (5 test cases)
Part 3 - BFS:            25/25 points (5 test cases)
```

```
-----
TOTAL SCORE:              60/60 points
-----
```

```
Note: Dockerfile configuration worth 40 points is graded separately
      Total project points: 100
```

```
=====
chang@chang-pc:~/docker-file/project1$ █
```